CS427 Coding Agent 项目报告

基于 mini-swe-agent 的工具、记忆、检索与修复工作流

2026-10-19 · xiaofei9289

https://github.com/xiaofei9289/2025Fall_CS427_agent

摘要

本项目在 mini-swe-agent 上扩展了一个面向 GitHub Issue 修复的 Coding Agent：保留“模型输出动作 → 环境执行 → observation”的简单循环，增加文件/检索/测试工具、可审计的上下文记忆、Python AST 代码检索、证据驱动的修复工作流，以及可取消/恢复的持久化任务服务。重点不是堆功能，而是把配置、题目、源码快照、轨迹、补丁和官方评分关联起来，使每次结果都能复核。

结论先行：工程实现和可复现性较强，离线 localization 有明确信号；但 2026-10-19 的 A/B/C p2final（A 9/10、B 7/10、C 7/10，C 仅 3 次 memory_search，C−B=0）是一次有效 run，却不是检索的因果实验。当前不能声称 memory/search 提升 SWE-bench；repair workflow 也还没有在线对照。报告因此把“已验证的工程能力”和“尚未验证的性能收益”明确分开。

1. 系统设计

系统由六层组成：Agent 循环、工具注册表、记忆策略、代码检索、修复工作流和任务服务。所有动作仍遵循统一协议：一次只允许一个 bash 或 tool 动作；工具返回 {output, returncode}；observation 以同样格式回传。这样轨迹可以线性审计，不同工具不会改变 Agent 的基本控制流。

模块	核心实现	设计目标
Agent 循环	default.py / MemoryAgent / RepairAgent	统一 action/observation；预算与终止语义
工具	tools/ + 注册表	容器/本机一致；路径限制、输出上限、错误回传
记忆	memory/history.py、memory/abc/	完整轨迹保留；摘要 + 可检索历史；预算可审计
检索	retrieval/repository.py 等	AST 索引、BM25、context、repo_brief、引用扫描
修复工作流	agents/workflow.py、repair/	LOCALIZE→REPRODUCE→PATCH→VERIFY→FINALIZE
服务	service/ + FastAPI	任务队列、独立工作副本、取消/恢复、checkpoint

设计上最重要的三个约束是：目标环境内执行、源码/配置冻结、失败可解释。检索索引在 /testbed 内执行；实验启动器冻结 src 并校验 digest；预算违规、空补丁、评测错误、缺失轨迹都会进入报告，而不是被静默丢弃。

2. 工具扩展

工具层覆盖读写、行选择读取、多文件读取、文本搜索、replace、定义查找、代码检索、引用扫描、测试执行和 TODO。所有容器工具通过 env.execute 执行，并在返回前限制输出长度；路径操作经过工作区校验。

工具	用途	环境路径	测试
read_file / write_file	基础读写	env + host	单测
refined_read_file	按行/范围/head/tail/around 读取	env + host	单测
read_many_files	多文件、单文件行数/总字符预算	env + host	单测
search_file_content	文件内文本/正则搜索	env + host	单测
replace	精确文本替换	env + host	单测
find_definition	AST 精确符号查询	env + host	Docker slow
code_search / repo_map / code_context	BM25、仓库地图、预算化片段	env + host	Docker slow（新增）
find_references	调用/导入/基类轻量扫描	env + host	Docker slow（新增）
run_tests	执行验证命令并记录结果	env	单测

仍需补的作业要求是“工具有效性”的正式 ablation：当前证明的是工具可用、容器兼容和单测覆盖，不是每个工具都能提升解决率。建议按 baseline / file-tools / retrieval / retrieval+brief 四组做 10 题 × 2 轮，指标包括 resolved、空补丁、官方文件读取/编辑/验证和成本。

3. 记忆：A/B/C 设计与结果

A/B/C 使用同一模型、10 题、400 步、每题 10 美元和 8000 估算 token 预算：

Arm	策略	超限后	memory_search
A	window	丢最旧完整 turn	无

B	summary	丢最旧 turn + 证据摘要	无
C	summary_search	同 B	有；另含 C-only 指令与 gate

p2final 是完整 generation + official grading 的有效 run：

Arm	Resolved	Empty	Cost	Calls	Search calls
A	9/10	1	$0.3046	1156	0
B	7/10	1	$0.5413	538	0
C	7/10	0	$0.6526	635	3

- C 只有 3 次 memory_search；7 道题没有检索。C−B=0，B−A/C−A 均为 −0.2。
- C 的指令、gate 和 provider 兼容层共同改变了 treatment，C−B 不能解释为“检索效应”。
- gate 只拦 replace/write_file 和含 COMPLETE_TASK 的提交；bash 编辑路径仍可绕过。
- 历史 50 题 baseline 42/50、memory-on 40/50，成本约 7 倍；单轮方差 37/40/42，2 题差异不可解释。

3.1 Sham-search 控制（2026-10-19）

为把“有用检索”与“只是多了一个工具/提示/gate”分开，增加 C-sham 控制：同名、同 schema、同描述、同 gate，但 memory_search 始终返回 No matching messages，retrieval_tokens=0。C-real 与 C-sham 使用同一源码、5 题、同一预算和提示，只有返回内容不同。

C 变体	Resolved	Cost	Calls	Search calls
real	4/5	$0.1802	185	5
sham	4/5	$0.3599	294	8

- 两者解决同一组任务（11333、14855、15315、8721），都失败 11532；没有 real-only 或 sham-only。
- sham 更贵、检索更多，说明无内容时模型会重试/重读；这不是检索有效的证据。
- 5 题没有统计功效；只能说明当前 C 在这组题上没有可见收益，不能证明检索普适无效。

因此当前结论是：memory 基础设施可运行、可审计，但没有证明性能收益；A（window）在本轮最好。sham 控制也没有显示检索收益。若继续研究，需要覆盖 bash 写操作的 gate、按“证据确实被挤出”选任务、以及 10 题 × 2–3 轮的 C-real/C-sham 对照；当前不应继续 50 题 memory 实验。

4. 代码检索

检索层使用标准库 AST 构建符号索引，遵守 Git ignore，跳过符号链接、大文件和解析失败文件；BM25 用于文件和符号排序；repo_map 输出预算化符号清单；code_context 进一步给出路径、行号、签名和代码片段；repo_brief 从 issue 中抽取路径、堆栈帧和反引号符号，在任务开始时注入；find_references 做轻量调用/导入/基类扫描。大仓库索引按根路径和 mtime/ctime/size 指纹缓存，可用环境变量关闭。

方法（50 题离线）	Recall@5	Hit@5	MRR
文件级 BM25	0.534	0.60	0.467
符号级 BM25	0.531	0.60	0.437
上下文选择	0.691	0.76	0.560
repo_brief	0.645	0.68	0.488

5 题 repo_brief 的 0.88/1.00/0.90 在 50 题回落到 0.645/0.68/0.488，是明显的小样本过拟合。更重要的在线证据来自 gate 审计：5/5 官方文件进入首轮 top-10、5/5 被读取、4/5 被编辑、3/5 解决。瓶颈是“检索后收敛/编辑/验证”，不是“找不到文件”。Django 2,927 文件冷启动约 9.4–9.6 秒，缓存命中约 0.6–0.7 秒。

5. 修复工作流与服务

RepairAgent 在 DefaultAgent 外增加 evidence ledger 和提交门：记录测试运行、diff 指纹、失败 ID 和验证状态；strict 模式下未验证提交会被阻止。run_tests 工具把测试结果写入结构化 observation；默认 workflow_enabled=false，不改变旧行为。服务层提供 FastAPI、SQLite 队列、独立 worker/工作副本、取消/恢复、checkpoint 和候选补丁原子保存。

这部分目前只有单测/台式机 checks 和服务生命周期验证，没有 workflow on/off 的 SWE-bench 对照。它能说明“可以这样组织修复流程”，不能说明“提高了解决率”。服务端配置测试通过也不等于官方隐藏测试通过。

6. 可复现性与测试

- 本机：690 passed, 18 skipped, 52 deselected；Ruff 通过。
- 台式机：checks 419 passed, 11 deselected；slow gate 9 passed。
- 所有 SWE-bench run 记录 source fingerprint、题目列表、配置和官方评分；实验目录保留历史结果。
- 新增 tests/tools/test_retrieval_docker.py，覆盖 code_context / find_references 的容器路径（slow）。

```
pytest -q
pytest -q -m slow
python scripts/verify_v2_baseline.py
```

边界：Mac 本机没有 Docker daemon；provider 会出现空/截断响应和原生 DSML 格式；这些必须在台式机端到端运行中验证。单测全绿不等于官方评分通过。

7. 总体评估

结论	证据	判断
平台可复现	freeze/manifest/audit/官方评分/688 测试	支持
离线检索有信号	50 题 context selection 0.691 vs 0.534	支持（仅离线）
memory/search 提升解决率	p2final C−B=0；C 仅 3 次 search；sham 4/5 vs 4/5	不支持
repair 提升解决率	只有单测/checks	未验证
工具提升效果	工具可用且兼容	未做 ablation

工程评分应看“可运行、可复现、可解释、边界诚实”；研究评分应看“多轮、held-out、在线、效应量”。本项目前者强，后者仍弱。当前最有价值的资产是冻结/审计基础设施、50 题 localization 集和失败归因，而不是某个单轮成绩。

8. 下一步

- P0：最终报告与交付物纳入 p2final 负结果；不再声称 memory/search 提升。

- P0：memory 以负结果收尾；sham 5 题无收益，不再跑 50 题 memory 对照。
- P1：工具组 ablation（10 题 × 2 轮）；没有增益的工具只保留工程价值。
- P1：retrieval 四臂在线 gate（baseline / brief_only / retrieval / retrieval_gate），重点看检索后编辑/验证。
- P1：repair workflow off/on 5—10 题，测 empty patch、verified patch、resolved。
- P2：拆 repository.py（782 行）、把 provider 兼容移到 adapter、统一文档与测试数字。

附录：关键产物

产物	位置
p2final 报告	experiments/memory_abc_20261019/PILOT_P2FINAL_20261019.md
sham 控制	experiments/memory_abc_20261019/SHAM_20261019.md
A/B/C 对照	experiments/memory_abc_20261019/comparison_pilot_p2final.md
5 题 localization	experiments/localization_verified5/
50 题 localization	experiments/localization_verified50/
检索归因	experiments/gate-20261017/retrieval_audit.md
检索设计	docs/retrieval.md
服务/部署	deployment/README.md